Revocation Is Not Termination

Counterexample-guided auditing of agent authority that outlives shutdown

Han (Harrison) Lee
Stanford University hanl@stanford.edu

Working paper, October 2026

Abstract

When an operator revokes an agent's authority, does the deployment stop? Not the model, but the deployment, with its queues, caches, retries, delegated credentials and gateways that learn about revocations late. I build an instrumented runtime in which authority can survive revocation through six interpretable mechanisms plus one withheld from the auditor, and an auditor that sees only partial telemetry, chooses experiments by expected information gain, minimises each surviving effect to a counterexample, and selects the cheapest set of interventions that blocks every path found while preserving independently authorised work. Across 146 deployments built from 30 workflow templates, static lineage revocation (cascade and cancel by declared lineage) closes 0 of 32 held-out deployments. Counterexample-guided containment closes all 32 and keeps 100% of independent work, where blanket shutdown destroys about 90%. Active probe selection finds the first surviving effect in 1.3 to 1.7 probes against 3 to 4 for random or checklist probing. The result I did not expect was that an oracle with full telemetry is overconfident on the withheld mechanism and wrongly claims containment a third of the time, while the partial-telemetry auditor runs the one experiment that exposes it and reports insufficient evidence. Live `gpt-5` planners avoided one leak only because the gateway resolved approvals by exact identifier. "Stopped" is a claim about schedules, and it should be audited like one.

1 Introduction

Shutdown is the oldest safety primitive we have and the least examined in deployment. The literature treats it as a property of the policy, asking whether the model will accept correction, whether it will resist being switched off, and whether training erodes that disposition [2, 3]. Those are the right questions about the model, and they are not the question an operator faces at two in the morning when a task has gone wrong and they press revoke. That operator wants to know whether the system around the model stops, and the system is larger than it looks. There is the queued job dispatched three ticks ago, the sub-agent still holding a delegated credential, the cached plan that quietly carries the revoked task's approval decisions into a colleague's reuse, and the retry that a supervisor re-created under a system identity because its parent was gone.

I came to this question from the other side. I had built an execution authority for agents, a gateway that holds credentials the agent never sees and issues single-use, signed authorisations bound to a task contract. Revoking a task there voids every descendant and every outstanding authorisation at the gate. The gate's own guarantee was clean. What I could not say was whether the deployment it sat inside shared that guarantee, because the deployment had a queue, a cache and a retry policy, and none of them were mine. The honest statement was that "the agent stopped" is a claim about the set of reachable schedules of a distributed system, and nobody had checked it.

This note reports a small, complete study of that claim, and it contributes four things. The first is a runtime with six interpretable ways authority can outlive revocation, each a flag on an otherwise correct reference implementation, with ground truth recorded as the full set of tasks whose authority each external effect relied on. The second is an auditor that works from partial telemetry and attribution at the tools rather than from internal logs, with a hypothesis space that separates known mechanisms from "something I have not modelled, on this kind of experiment." The third is counterexample-guided containment (CEGC), which combines record and replay, delta-debugging minimisation, blocking-set discovery, explanation, and a grouped optimiser over intervention sets that charges collateral damage to legitimate work. The fourth is a benchmark of 146 deployments with a held-out mechanism, together with the finding that richer telemetry made the auditor worse at knowing what it did not know.

2 Setting

The runtime has a planner, two workers, a shared plan cache, a retrying delayed queue, an authority service and a commit gateway between the workers and simulated external tools (a ledger, email, a CRM). Operators revoke tasks.

Every effect at a tool records its lineage, meaning the delegation chain of the credential presented, the origin of any carried approval, and the original lineage of a retried job even after the runtime relabels it. A violation is an effect whose lineage includes a task the operator had already revoked. The runtime never reads this record. It exists for scoring.

Table 1. How authority survives revocation in the testbed, and what blocks it. The last row is withheld from the auditor's model.

mechanism	how authority survives	blocking interventions
`enqueue_only`	gateway trusts the check made when the job was enqueued	`commit_check`, `cancel_queue_lineage`
`child_survives`	revoking a task leaves delegated credentials valid	`cascade_revoke`, `commit_check`
`cache_carries_auth`	a cached plan keeps the origin task's approvals, and reuse carries them	`strip_cache_auth`, `commit_check`
`retry_system`	a retry of an orphaned job is re-created under a system identity	`retry_lineage`
`worker_attested`	gateway trusts the worker's stale credential view, replayed by retries	`commit_check`
`async_gateway`	gateway learns of revocations from a delayed push	`sync_revocation`, `commit_check`
`batch_leader` (withheld)	coalesced jobs commit under the batch leader's authority	`split_batches`

Knowledge and authority are deliberately separated. A cached plan is knowledge, and reusing it under fresh authority is fine. A cached approval decision is authority, and it must not travel with the plan. A unit test pins down all four combinations, because the distinction is easy to state and easy to lose in an implementation.

3 The auditor

Observations. The auditor always sees its own revocations and every effect at the tools. Under *standard* telemetry it also sees some gateway, queue, cache and crash events with credentials, reasons and lineage stripped, and each exported event is dropped with probability 0.3. *Sparse* shows effects and crashes only. *Full* shows everything, including the gateway's allow reasons, and serves as the oracle baseline. Without lineage, a surviving effect is still attributable in two cases. A task acts after its own revocation, or a probe task acts with authority its own grant lacks. That is why "reuse the revoked task's plan" is run with a task that has no send authority.

Probes. There are ten experiment types, each timed against the template's undisturbed timeline. They are revoking at the commit tick, delaying the revocation notice, crashing and restarting a worker with a stale credential snapshot, revoking after enqueue, revoking between dequeue and commit, failing a step and stopping its worker before revoking ahead of the retry (in queued and direct variants), reusing the plan without send authority, reusing it under independent authority, putting two tasks' jobs in one batch, and an early-revocation control. Probe cost counts the run itself, extra planning tasks, crashes and restarts.

Belief. There are 64 hypotheses, one per subset of the six known mechanisms, plus catch-alls OTHER_k, one per experiment kind k, each meaning "something unmodelled shows up on experiments of kind k." Likelihoods come from simulating each probe under each hypothesis with Monte Carlo over timing jitter. Telemetry loss is handled exactly, as a product over observed items in which each item is visible with probability $1 - 0.3^m$, where m is the number of events that could have revealed it. Splitting the catch-all by experiment kind was the decision that mattered most. With a single OTHER, every probe the known model explained well drove its mass to zero, and the auditor never ran the one experiment that exposes the withheld mechanism. Probe value is expected information gain in bits, plus α times the probability of an attributable violation, minus β times cost, with α and β tuned on the development split only ($\alpha = 2$, $\beta = 0$). A depth-two lookahead policy was implemented for comparison.

Containment (CEGC). Every counterexample goes through the same loop. The auditor first replays the exact schedule with its recorded timing, then minimises it with DDMIN over the removable conditions, namely extra tasks, crashes, restarts, injected failures, timing knobs and workflow steps, and replays the minimised schedule under each single intervention to learn which ones block it. Explanation comes next, and it is deliberately strict. A known mechanism explains the counterexample only if, simulated alone, it reproduces it and every intervention that blocked it in replay also blocks it in the model, and otherwise the counterexample is marked unexplained. The path (revocation, cache, delegation, propagation, restart, queue, batch, retry, gateway, effect, as applicable) is added to an authority graph, and a grouped optimiser then enumerates intervention sets, keeps those that block every known counterexample in replay, and minimises static cost plus λ times the observable loss of independent work on a collateral suite, with a weighted min-cut on the graph computed for comparison. Finally the auditor reruns nearby variations (revocation tick ± 1 and ± 2, notice delay, dispatch latency) and loops on any survivor.

Claims. With white-box access the auditor finishes with a bounded exhaustive check over 2,000 to 3,100 schedules

per template (every revocation tick, notice delays of 0, 1 and 6, dispatch latency of 1 and 2, four task mixes, and crash, restart and transient-failure placements, with jitter off) and reports `closed_within_bound` or a counterexample. Otherwise it never claims proof. It reports `no_counterexample_within_budget` only when the posterior mass on "some known probe still exposes revoked authority under the chosen interventions," plus all OTHER mass, is below 0.05. It reports `insufficient_evidence` when a counterexample stays unexplained or OTHER mass exceeds 0.3, and `residual_risk` otherwise. "Closed" is a statement about this grid of this simulator and nothing more.

4 Benchmark

Thirty workflow templates are built from seven topologies over seven business domains and paired with deployments, giving 146 audited instances. The *dev* split (96) has direct pairs, queued sends, queued pairs and mixed chains, and each template gets the reference plus three single-flaw and two two-flaw deployments. The *gen* split (32) has new topologies (the planner acting directly, sub-delegation through a second worker) with two- and three-flaw combinations. The *hard* split (18) has fan-out queues with the withheld `batch_leader`, alone or with one known flaw, plus the reference. The budget is ten probes per audit. Schedules are generated without model calls. Section 6 swaps in a frontier planner.

5 Results

Table 2. Probe policies at budget 10. *Probes to first* counts probes to the first surviving effect (misses count as 11). *Coverage* is the share of reachable mechanisms exposed. *False claim* is the share of "no counterexample within budget" claims refuted by the bounded check. *Closed* is the share of deployments closed within the bound after the chosen interventions.

split	policy	probes to first	coverage	false claim	insufficient evidence	closed
dev	random	3.00	0.79	0.00	0.12	0.86
dev	fixed checklist	4.03	0.77	0.00	0.03	0.85
dev	**active (1-step)**	**1.71**	**0.90**	0.00	0.09	**1.00**
dev	lookahead (2-step)	1.56	0.89	0.00	0.10	1.00
dev	active, full telemetry (oracle)	1.63	0.90	0.00	0.01	0.99
gen	random	2.66	0.54	0.00	0.34	0.59
gen	fixed checklist	3.22	0.71	0.00	0.12	0.88
gen	**active**	**1.28**	**0.78**	0.00	0.19	**1.00**
gen	lookahead	1.22	0.81	0.00	0.19	1.00
gen	oracle	1.19	0.86	0.00	0.00	1.00
hard	random	6.33	0.54	n/a	0.33	0.72
hard	fixed checklist	7.42	0.25	0.50	0.00	0.33
hard	**active**	5.17	**0.96**	**0.00**	0.61	**0.94**
hard	lookahead	4.50	0.88	0.00	0.50	0.83
hard	oracle	4.92	0.75	0.33	0.33	0.67

Table 3. Containment against shutdown baselines, using the active policy's chosen interventions. Violation rate is the share of bounded-grid schedules with a surviving effect. Legitimate completion is measured on the independent tasks of the collateral suite. Latency is the maximum revocation-to-last-effect in ticks.

split	containment	closed	violation rate	legit. completion	cost	latency
dev	parent-only revocation	0.22	0.38	0.91	0.00	7.3
dev	static lineage (cascade + cancel by lineage)	0.28	0.17	0.91	1.07	5.9
dev	blanket shutdown	1.00	0.00	0.09	11.62	0
dev	**CEGC**	**1.00**	**0.00**	**1.00**	1.18	0
gen	static lineage	0.00	0.33	0.76	1.19	12.1
gen	**CEGC**	**1.00**	0.00	1.00	2.09	0
hard	static lineage	0.33	0.07	0.97	1.12	3.4
hard	**CEGC**	0.94	0.00	1.00	1.08	0.06

Active selection beats random and checklist probing on time to first finding, by a factor of two to two

and a half, and on coverage, across all three splits. The checklist is worst on *hard* because it spends its budget on experiments the withheld mechanism never touches.

Missing telemetry costs little for discovery and a great deal for diagnosis. Detection relies on attribution at the tools, not on internal logs, so the oracle is only slightly faster to a first finding. Under standard telemetry, however, a minimised counterexample often fits two or three mechanisms at once, and the oracle is far more willing to claim containment (claim rate 0.82 against 0.60 on *dev*).

Richer telemetry made the auditor overconfident on the withheld mechanism. On *hard*, the oracle identifies the known flaw at once and installs queue cancellation, which happens to hide the batch path from later probes. It then claims containment wrongly a third of the time. The active auditor under partial telemetry runs the batch experiment, finds an effect no known mechanism explains, contains it with `split_batches`, and reports insufficient evidence instead of a claim. At budget 20, claim rates rise everywhere (*dev* 0.90, *gen* 0.81, *hard* 0.33) and false claims stay at zero for the active policy on every split, while the checklist (0.40), lookahead (0.25) and oracle (0.14) still make them on *hard*. I read this as the central result. An auditor's calibration about its own model is worth more than its telemetry, and more telemetry can buy a confident wrong answer.

Static lineage revocation misses every path absent from the declared lineage, namely stale gateway views, worker attestations, cached approvals, retries under a system identity, and batches. It closes 0 of 32 generalisation deployments. CEGC closes all 32. This matters because cascade-and-cancel is what a careful engineer would implement first, and it is what most agent frameworks mean by revocation.

Blanket shutdown contains everything and destroys about 90% of independent work. Selective containment keeps all of it at roughly a tenth of the static cost. The question is never whether one can stop everything. It is whether one can stop the right things.

Depth-two lookahead is not worth it. It finds the first counterexample slightly sooner but has lower coverage and closure on *hard*. One-step selection is the default.

Grouped optimisation against min-cut. On the same discovered paths, the weighted min-cut blocks 99 to 100% of known counterexamples. Its weakness is cost. It charges a shared intervention once per edge, so it chooses combinations of narrower interventions (2.98 against 2.09 on *gen*) and is closed less often within the bound (0.75 to 0.84 against 1.00). When measured costs tie, the grouped optimiser prefers fewer interventions, which favours a single `commit_check` over pairs such as `cancel_queue_lineage` with `sync_revocation`, and the broader check then also covers schedules nobody tested.

One path no probe exercised. A failed direct step whose retry replays the worker's stale attestation. Only the bounded check found it, and only in the first round of experiments. The probe catalogue now includes direct-step retries. This is the argument for running the bounded check whenever white-box access exists.

Commit-time checking, not cancellation. The original proposal paired commit-time checks with scoped queue cancellation. In the benchmark, every deployment where the enqueue-only flaw was reachable ended with `commit_check` and never with cancellation. Cancellation is cheaper, but it cannot stop a job that was dequeued before the revocation, and the in-flight variations of the final containment step find that case every time.

Containment can outrun diagnosis. In the demonstration deployment with both `enqueue_only` and `cache_carries_auth`, parent-only shutdown leaves a colleague's task running on the revoked task's approvals (0 of 2 legitimate completions). A commit-time check alone would block those effects and the colleague's work with them. The grouped optimiser measures that collateral and adds `strip_cache_auth`, so the colleague keeps the plan, loses the borrowed permission, and completes under its own authority. The auditor fixed the cache flaw with a posterior of only 0.28 on that mechanism, because once stripping was in place no experiment could tell. That is a feature for an operator and a warning for anyone who reads the posterior as an explanation.

6 Real planners

A second planner replaces template step content and cache adaptation with a frontier model under a strict JSON schema. Topology stays fixed in orchestration code so ground truth remains exact. When the cache is flawed, the cached plan shows `approval_ref` fields, and the runtime honours whatever the adapter returns, so whether reuse carries authority forward becomes a measured property of the model rather than an assumption. With `gpt-5` (low reasoning effort, four templates, eight distinct adaptations, fifteen calls), the model never copied an approval verbatim. It rewrote every `approval:A:<step>` into `approval:<new task>:<step>`, inventing an approval in the new task's name instead of carrying the revoked task's forward. Because this gateway resolves approvals by exact identifier, a made-up reference matched nothing and the flawed cache never leaked (0 violations, against 2 per schedule with the template planner). I read that as luck, not a safeguard. A gateway that matched approvals by step name, or trusted

any well-formed reference, would accept the rewritten approval and would also grant the reusing task a send it was never authorised for. With one model, one prompt wording and eight adaptations, this shows the copy-everything assumption can be wrong, not a rate.

7 Related work

AI control [1] designs protocols that remain safe when the policy is adversarial, but its guarantees are stated per episode and end at the monitor's decision, which is where this note begins. Corrigibility and the off-switch game [2, 3] ask whether the agent will let itself be stopped, whereas I ask whether the deployment lets the operator stop it, with an entirely cooperative model. Capability-based security [4] supplies the vocabulary of attenuation and revocation. Authenticated delegation for agents [5] and agent infrastructure [6] argue for the identifiers and logs that make attribution possible, and this note is one way to test whether a given deployment honours them. Counterexample-guided abstraction refinement [7] and delta debugging [8] give the containment loop its shape, and expected-information-gain experiment selection [9] gives the probe policy. Trace-level security constraints for agents [10] govern what an agent may do within a run, whereas the hazards here arise between runs and between components.

8 Limitations, and what I would do next

The hypothesis space is six hand-written mechanisms. The withheld-mechanism split shows the auditor can notice when reality is outside its model, but only because the probe catalogue happened to contain an experiment that triggers it. It never invents probe types. The intervention catalogue is fixed and already contains `split_batches`, replay assumes recorded timing, collateral is measured on a small fixed suite, closure covers one discretised grid with jitter off, and revocation is operator-triggered only. Each audit spends 400 to 700 sandbox replays on minimisation, blocking sets, optimiser feasibility and collateral measurement. Replays are cheap in a simulator and would not be against a real deployment.

The quantity I most want out of this work is not a closure rate on my own simulator. It is *revocation-to-last-effect latency* and *stale-authority exposure* measured on real agent frameworks, as a scaling quantity rather than a testbed artefact. That means instrumenting open orchestration frameworks, treating their queues, caches and retry policies as the hypothesis space, and asking whether exposure grows with autonomy horizon the way task length does. If it does, "can we stop it" acquires a trend line, and trend lines are what governance can act on.

References

[1] R. Greenblatt, B. Shlegeris, K. Sachan, F. Roger. AI Control, Improving Safety Despite Intentional Subversion. ICML 2024. arXiv 2312.06942.

[2] N. Soares, B. Fallenstein, E. Yudkowsky, S. Armstrong. Corrigibility. AAAI Workshop on AI and Ethics, 2015.

[3] D. Hadfield-Menell, A. Dragan, P. Abbeel, S. Russell. The Off-Switch Game. IJCAI 2017.

[4] M. S. Miller. Robust Composition, Towards a Unified Approach to Access Control and Concurrency Control. PhD thesis, Johns Hopkins University, 2006.

[5] T. South, S. Marro, T. Hardjono, R. Mahari, C. D. Whitney, D. Greenwood, A. Chan, A. Pentland. Authenticated Delegation and Authorized AI Agents. arXiv 2501.09674, 2025.

[6] A. Chan et al. Infrastructure for AI Agents. Transactions on Machine Learning Research, 2025. arXiv 2501.10114.

[7] E. Clarke, O. Grumberg, S. Jha, Y. Lu, H. Veith. Counterexample-Guided Abstraction Refinement. CAV 2000.

[8] A. Zeller, R. Hildebrandt. Simplifying and Isolating Failure-Inducing Input. IEEE Transactions on Software Engineering 28(2), 2002.

[9] D. V. Lindley. On a Measure of the Information Provided by an Experiment. Annals of Mathematical Statistics 27(4), 1956.

[10] Invariant Labs. Formal security analysis of AI agents via trace and data-flow constraints. ICML 2024 workshop.